Subject: Computer Science

Exploring supervised learning

Research question: “How can K-Nearest Neighbors help us predict a Major League Baseball player’s on-base percentage?”

IB Personal Code: ljm852
Word count: 3998

Table of contents

Introduction

In Major League Baseball (MLB), the importance for predictive analytics has always been key. Every team looks for an advantage that could make the difference between winning and losing. The ability to predict players' on-base percentage (OBP), which is how often a batter gets on base, accurately is crucial in scoring more runs than the opposing team *(Cclapp, 2023)*. In this exploration, I will dive into supervised learning models, more specifically K-nearest Neighbors (KNN), to explore their potential in enhancing our prediction of MLB players' performances. Supervised learning models have repeatedly shown their ability in real-world applications such as categorizing diseases given the symptoms *(Bansal, 2019)*; in this study, players' statistics can be analyzed by employing these models to identify patterns and relationships in large datasets. By using the power of machine learning algorithms, these models have the potential to reveal hidden insights. Thus, this leads to the research question:

"How can K-Nearest Neighbors help us predict a Major League Baseball player's on-base percentage?"

This topic is worthy of investigation because I want to improve from human statistics methods and evaluations. The conventional methods often miss the possible connection of variables, and supervised learning offers an approach that surpasses human limitations and reveals latent correlations *(Bechtold, no date)*. On a broader scale, the methodologies gained from this study could enhance existing models in other sports domains, which refines predictive models and decision-making processes in the athletic world.

Generally, this investigation aims to predict an MLB player's OBP by the end of the season (around October) based on their statistics in April. The K-Nearest Neighbors Regression (KNNR) model is chosen over other algorithms like linear regression due to its intuitive approach of using nearby observations to predict a target variable (GeeksforGeeks, 2018). The process begins by merging the dataset containing player statistics from April with the dataset containing their end-of-season OBP. Then, this merged dataset will be labeled with the statistics in April as input features and the end-of-season OBP as the target variable. Next, the dataset will be divided into a training set used to train the KNNR model and a testing set used to evaluate the model's performance on unseen data. After the training process is completed, the model will be capable of predicting the end-of-season OBP for a new player based on their statistics in April.

What is supervised learning?

Supervised learning models are a primary type of machine learning model. These models are trained using labeled datasets, one training set and one testing set, where both the input features and the corresponding target variables are predefined. By learning the relationships between the input features and the target variables from the training set, the model can accurately predict outcomes for the testing set *(IBM, 2023)*. Supervised learning can be divided into two sub-categories: classification and regression. Classification involves predicting a discrete label or category for given input data, such as determining the animal shown in an image. Regression, on the other

hand, involves predicting a continuous value, such as an athlete's performance based on their previous statistics *(Crashcourse, 2019)*.

Data selection

In this study, April was specifically chosen because it is the beginning of the regular season, ensuring that players are already in optimal condition after preseason training. Moreover, early-season evaluations can provide teams with useful information to allow them to make reasonable decisions about trades to enhance overall team performance as the season progresses. By analyzing statistics in April, teams can gain an early indicator of players' end-season OBP, which helps in strategic planning and roster management. The data for this analysis was sourced from *Baseball-Reference*, which provides labeled and accurate MLB statistics. This study focuses on data from the 2023 season to ensure that the model is built upon the most recent MLB statistics and rules, as changes in regulations can influence player performance.

Choice of programming language

Python was chosen for this study because it is widely recognized as the standard programming language for machine learning due to its libraries, ease of use, and strong community support. Additionally, Python is supported by Jupyter Notebook, which presents an ideal environment for structuring and exploring AI models.

K-Nearest Neighbors Regression (KNNR)

The KNNR algorithm is non parametric, meaning that this model does not assume any relationship between the input features and the target variable. I initially thought that working with machine learning algorithms would involve multiple steps of deciphering and understanding, but I found KNNR greatly intuitive, making it an ideal starting point into the field of machine learning, so I chose it. Additionally, unlike many other models, KNNR doesn't require an intensive training phase, which can save both time and improve efficiency.

In KNNR, data points are represented as vectors in an *n*-dimensional space, where *n* is the number of input features. Each data point is then classified based on its distance to other points within this space. These are the steps that the model undergoes when a new data point is introduced.

1. The model calculates the new data point's distance to all training data points using Euclidean distance *(Simplilearn, no date)*

For a given data point, x, its distance to a datapoint y can be calculated using the Euclidean formula:

$$\text{Distance}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Where x_i and y_i are the values of the ith input feature for x and y respectively, and n is the total number of dimensions.

Figure 1: KNNR model with 2 input features

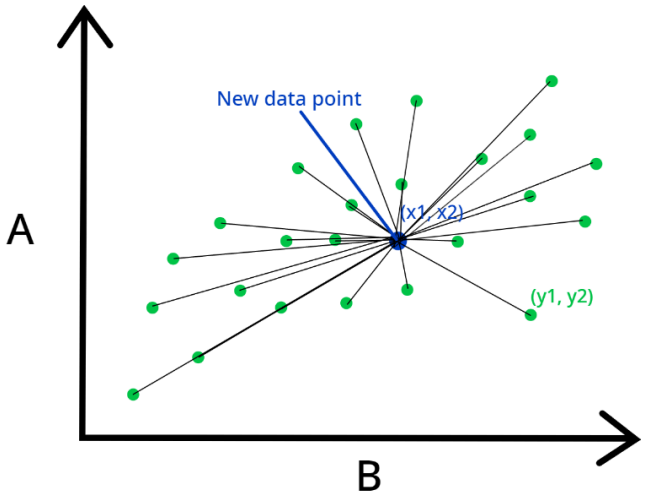

Diagram made by candidate

Applying the formula to calculate the distance here in 2 dimension gives

$$\text{Distance}(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

2. The model identifies the K-nearest neighbors to the new data point based on the distances calculated previously *(Simplilearn, no date)*

Figure 2: KNNR model with 2 input features when K is equals to 4

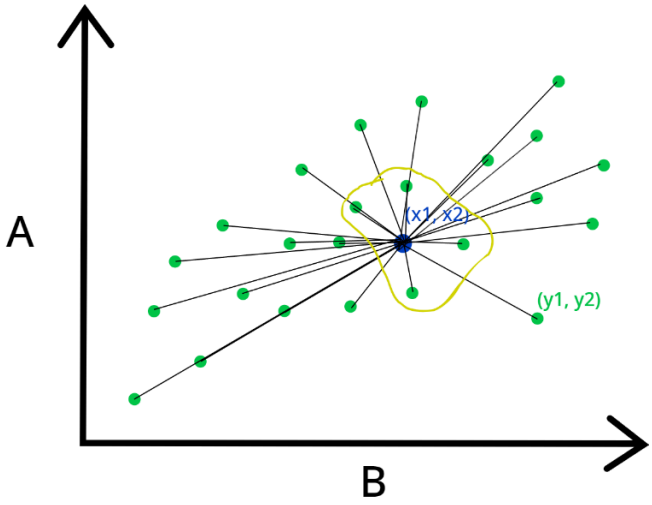

Diagram made by candidate

In this example, assuming K = 4, the model has identified the new point's 4 closest neighbors.

3. Take the average of the outputs of the K-nearest neighbors *(Simplilearn, no date)*
4. This average is the predicted result of the new datapoint *(Simplilearn, no date)*

Picking the K value is the most important step to ensure the most optimal supervised model performance. A graph will be plotted with K on the x-axis and the performance of the model on the y-axis. The K-value is then determined by observing and analyzing the curve corresponding to both training and testing sets. *(GeeksforGeeks, 2018)*

So how does the K value influence the model's performance?

A small K value can typically lead to overfitting, meaning that the algorithm fits too closely or even exactly to its training data, which results in a model that can't make accurate predictions from any data other than the training data. On the other hand, a high value of K leads to underfitting, meaning that the model approximates too much on the training data and cannot be generalized to predict new data. Thus, setting the K value to values such as one or a big number is often avoided; it is best to find the balance between overfitting and underfitting to build an accurate model. *(R-bloggers, 2022)*

Figure 3: Diagrams of the consequence of choosing a K-value

"How to Avoid Overfitting? | R-Bloggers." R-Bloggers, 6 Sept. 2022, www.r-bloggers.com/2022/09/how-to-avoid-overfitting/#google_vignette. Accessed 30 Sept. 2024.

However, the KNNR model has some downfalls. Since it calculates the distances separating each datapoint, the computational time is slow compared to other algorithms when applied to large datasets. Additionally, KNNR is sensitive to irrelevant features, so it weights all the input features equally *(Soni, 2020)*. For instance, if I included the type of phone that the MLB players use in my study, assuming it has no correlation to the players' performance, it would negatively impact my model's performance. Thus, it is important to consider the most relevant features to secure a reliable model.

Building the AI model

How did I extract the data off the internet?

The available data online was saved into an Excel spreadsheet and exported as a CSV file. The CSV files were then imported into Jupyter Notebook using the Pandas library in order to access and manipulate the files with Python.

What are the features I chose?

Figure 4: Table of possible features to choose from

Feature	Meaning
PA	Number of at-bats
HR	Number of homeruns
R	Number of runs scored individually
RBI	Number of runs batted in by a player
SB	Number of stolen bases
BB%	Percentage of a player gets on base without a hit
K%	Percentage of a player getting struck out
AVG	Batting average, measures a player's success at the plate
BABIP	Batting average on balls in play (hits in fair territory)
SLG	The total number of bases a player reaches per plate appearance
wOBA	A player's overall offensive contributions

	per plate appearance
BsR	The number of runs a hitter creates
WAR	A player's overall value to a team

Among the above, I chose R, AVG, and SLG. These data are clear indicators of how many times players reach the bases, which I think are perfectly suited for this study.

1. R tells us the number of runs they scored, thus represents the minimum amount of times players reached the bases
2. AVG tells us how well a player performs at each PA, thus how likely they are to reach the bases
3. SLG explains itself explicitly

Other features were either derived data, which introduce bias, or unrelated to this study.

Data distribution analysis

Before any analysis, it's important to clarify that higher values in each feature indicate better player performance. And a high OBP is what teams look for.

Figure 5: Frequency vs. OBP distribution graph

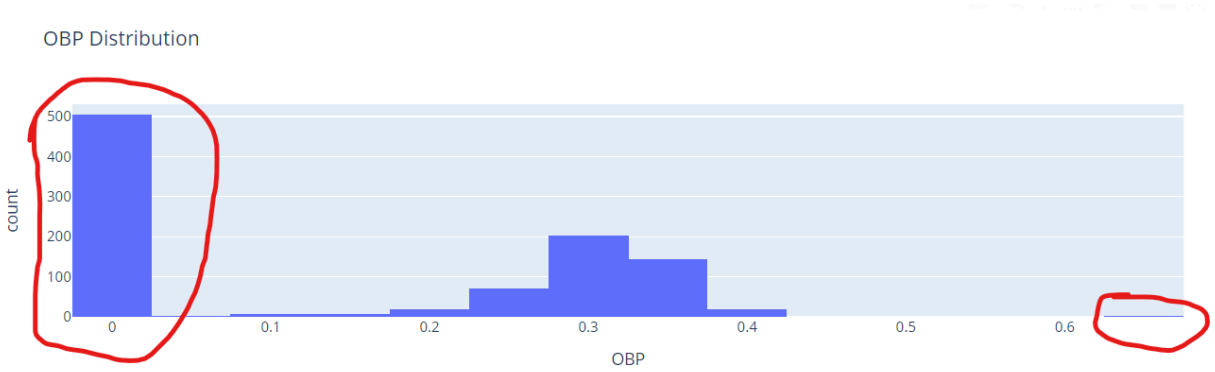

Diagram made by candidate

Starting from the OBP data distribution, it can be observed that approximately 50% of the data points are at 0 OBP, which are players who don't hit. If a model simply predicts 0 OBP for every player, it would already achieve around 50% accuracy, introducing significant bias, which is a systematic error that occurs due to wrong assumptions during the machine learning process. Hence, I removed all data points with 0 OBP to avoid this bias. We can also notice an outlier at OBP = 0.667. Indeed, this player is found to only have 2 plate appearances, which is not representative of their actual OBP.

Figure 6: Frequency vs. SLG distribution graph

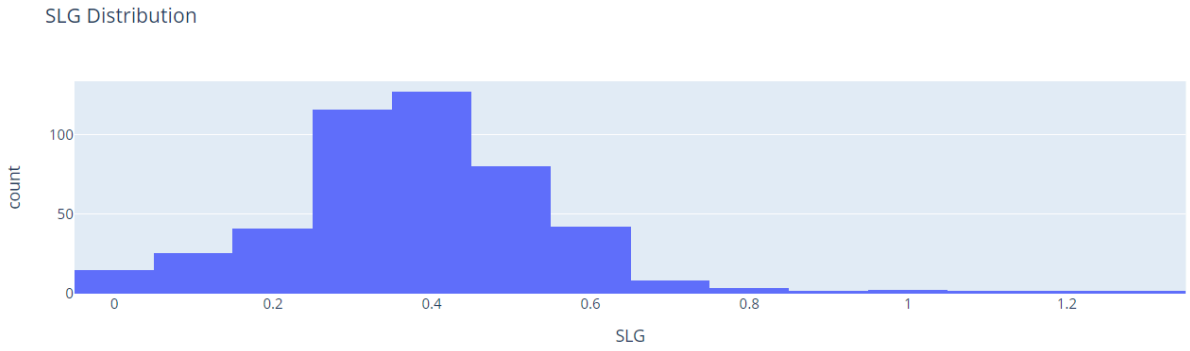

Diagram made by candidate

This distribution resembles a Gaussian distribution, meaning it is symmetric around the mean of the x-axis, with data near the mean being more frequent than data

further from it. But here the distribution is slightly skewed to the right: the peak of the distribution is shifted to the left, and the right side of the distribution fades away gradually. This indicates that the players are more concentrated on the lower side of the mean.

Figure 7: Frequency vs. AVG distribution graph

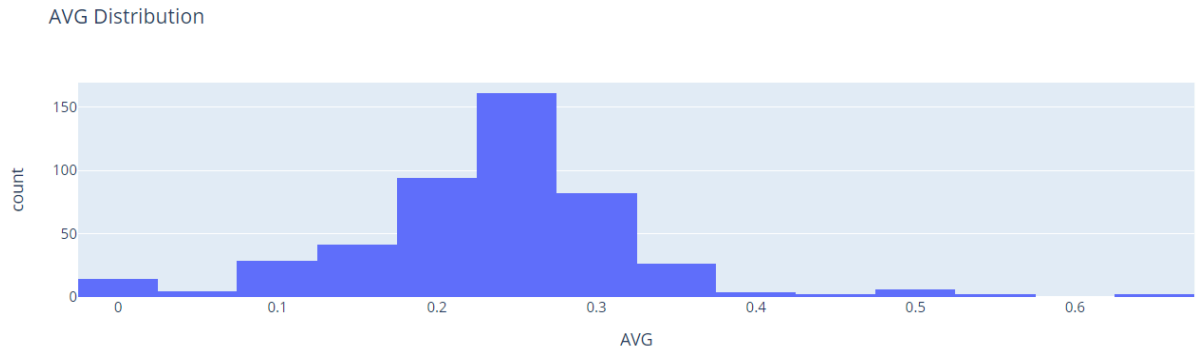

Diagram made by candidate

Similar to SLG, this is a Gaussian distribution with no skew, which means the left and right sides of the distribution about its mean are mirror images of each other. Most of the data points cluster around the mean, and the probabilities for values further away from the mean decrease equally in both directions. So this tells us that the players are concentrated around the mean AVG value.

Figure 8: Frequency vs. R distribution graph

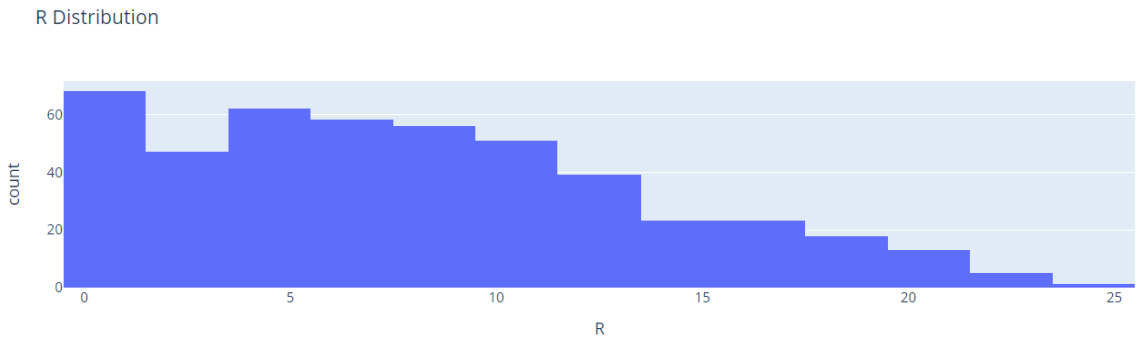

Diagram made by candidate

The R distribution reveals a decrease in count as R increases, this suggests that players with higher values of R are less common. The peak of the distribution is located towards the left side, indicating that most of the players are clustered around lower values.

Data preprocessing

I have 2 datasets: one from April and one from the end of the season; after checking for and removing duplicates to facilitate merging, I identified the common players among the 2 datasets, filtered both to include only these players, selected relevant input features, and then merged them. Following, I removed the data points with a 0 OBP value and I splitted the merged dataset into testing and training sets using 80% of the data to train the model.

All input features for the training and testing sets are then standardized, meaning their values are set to between -1 and 1. This can improve the performance of machine learning models by enabling faster convergence and more accurate predictions, as KNNR, which relies on Euclidean distances as discussed on page 7, benefits from smaller numbers to compute. *(Simplilearn, no date)*

What is Mean Squared Error (MSE)?

MSE will be used for the performance metric, it's commonly used to measure the performance of a regression model. The main idea is to compare the predicted results with the actual results.

As seen in the formula *(Stewart, 2023)*:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(x_i - y_i)^2 \text{ where}$$

n is the number of datapoints

x_i is the actual value for the ith data point

y_i is the predicted value for the ith data point

Starting off by choosing a random value of K = 4 to train the model to check the MSE, it gives a surprising low value of 0.00182 $(3\ s.f)$, indicating a high model accuracy.

Choosing the best value of K using the elbow method

The elbow method is a graphical technique used to determine the optimal value of K in KNNR. The "elbow" point on the graph, a shift in the gradient of the curve, marks the most suitable K, representing the point where increasing K further does not significantly enhance the model's performance and may even degrade it due to over-smoothing.

When plotting the MSE against different values of K, the elbow point is where the error rate stops decreasing sharply and begins to slow down. This implies that the added information from increasing K beyond this point does not provide large improvement and may potentially capture noise instead of the true relationships, hence overfitting. I have decided to evaluate different values of K ranging from 1 to 20, with 20 selected as an upper limit to help prevent underfitting and identify the elbow point (Saji, no date).

Figure 9: KNNR performance graph

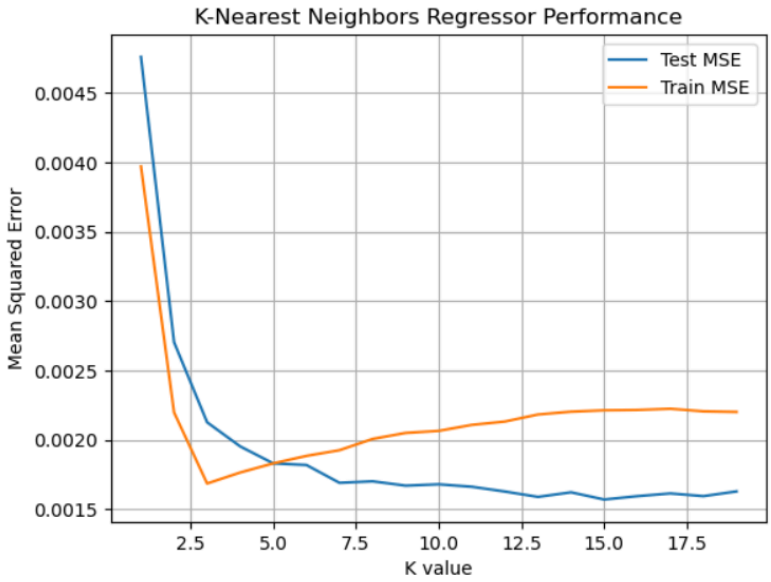

Diagram made by candidate

Observing the test MSE curve, the elbow is located around K = 3. The left side of the elbow represents the underfitting region and vice versa *(Super Data Science, 2023).*

This optimal K balances the trade-off between bias, and variance, the error caused by the model's sensitivity to fluctuations in the training data, leading to a model that generalizes well to unseen data. *(GeeksforGeeks, 2020)*

I also need to compare the train and test MSE to check for overfitting and underfitting. Here, the train MSE decreases before K = 3 and then increases afterwards. Usually a very low MSE on the training set can be an indicator of overfitting, it can be seen as remembering the results of the training set. However, in this case, the graph shows that both the train and test MSE values are low at K = 3, and the increase in train MSE for higher values of K suggests that the model starts to underfit as K increases.

This means that K = 3 is not necessarily overfitting if the test MSE is also low and stable. *(Saji, no date)*

In conclusion, K = 3 is a good choice using the elbow method, showing that the model performs well on both the training and testing sets. This gives a MSE of 0.00208 ($3\ s.f$) which is higher than 0.00182 ($3\ s.f$) from before when K = 4. However, with K = 3, the model has a lower chance to underfit or overfit. Thus, K = 3 will be used for the rest of this exploration.

Curse of dimensionality

The curse of dimensionality occurs when adding more input features, or dimensions, causes the amount of data required for the model to generalize accurately to grow exponentially, increasing the risk of overfitting and decreasing model performance. As dimensions increase, data points become more spread out, making it harder to find relationships between them, while computational complexity rises and distance metrics become less reliable *(Analytics Vidhya, 2021)*. This is a problem especially in KNNR since it relies only on the distance metrics in any dimension. Given KNNR also assumes all features to be equally relevant, it results in higher inaccuracies if irrelevant features are included. *(GeeksforGeeks, 2024)*

To mitigate this issue, a standard approach is to add more data as input features increase, filling out the dimensions, reducing the distance between points, and thus improving the model's accuracy.

Here is a visual example of this approach:

Figure 10: 10 data points in 1 dimension

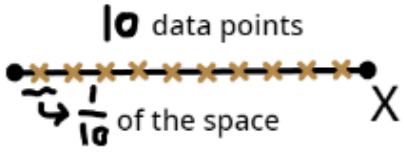

Diagram made by candidate

In this diagram, we have 10 data points filling up 1 dimension space with equal distance between them.

Figure 11: 100 data points in 2 dimensions

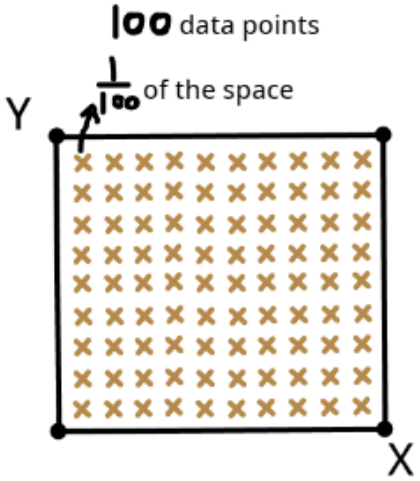

Diagram made by candidate

After adding a dimension, the number of data points increased from 10 to 100 to fill up the space to keep the same distance between data points, illustrating how the amount of data required grows rapidly with each additional dimension to mitigate the effects of adding a new input feature. *(Udacity, 2015)*

For this investigation, I do not have more available data since the number of MLB players is limited. Instead, I will find the most relevant features where the MSE is the

lowest and see if the model's performance will be better if I only select 2 features instead of 3.

Finding the most relevant features

Starting by finding all possible features, those that I haven't considered previously in the study, of combinations of 2, I can observe which combination has the lowest MSE. Here are the features I will use:

1. HR
2. R
3. RBI
4. SB
5. AVG
6. SLG

The combination ('R', 'AVG') has the lowest MSE at 0.00171 (3 s.f.), suggesting these are the 2 most relevant features for an accurate model. However, the result varies each time, so I will identify which two features appear most frequently among the top combinations.

After repeating the process of finding the best combination 60 times to avoid bias, AVG and SLG are found to be the 2 most frequent relevant features.

Figure 12: Bar chart of the frequency of each feature

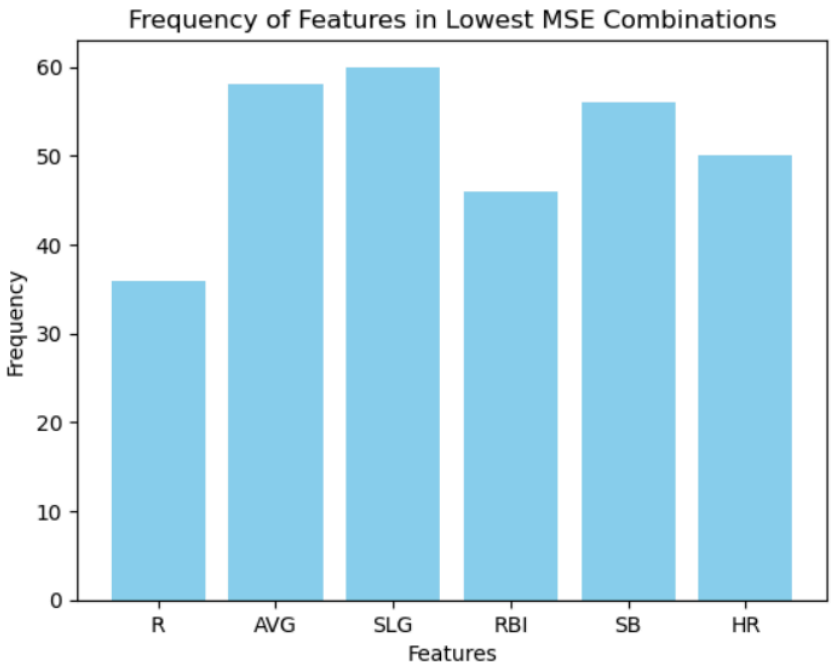

Diagram made by candidate

This result confirms my initial choice and evaluation of the relevant features.

Figure 13: SLG against AVG colored scatter plot

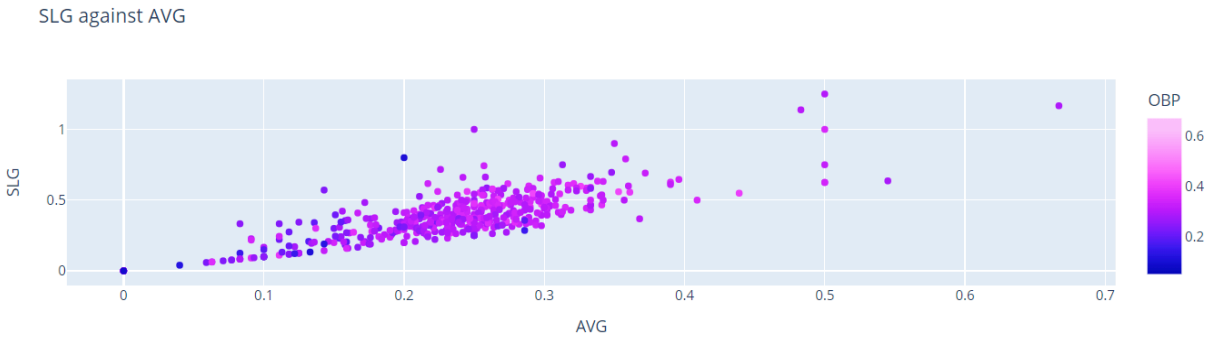

Diagram made by candidate

From the graph, players with the highest OBP are generally clustered around average values of both AVG and SLG, specifically with AVG ranging from 0.2 to 0.3 and SLG ranging from 0.3 to 0.5. The color of the scatter plot, which represents OBP, tilts

towards purple as both AVG and SLG increase, indicating higher OBP values. This color intensification reaches a peak, highlighting the optimal combination of AVG and SLG for maximizing OBP. Beyond this peak, the colors gradually shift from light purple (high OBP) to deep purple (lower OBP), suggesting a slight decrease in OBP even as AVG and SLG continue to rise. This suggests that while increases in AVG and SLG contribute positively to OBP, there is a point beyond which additional increases may not continue to boost OBP.

Comparing MSE of 2 features and 3 features

The MSE of AVG and SLG is found to be 0.00164 ($3\ s.f$), whereas the MSE of R, AVG and SLG is 0.00173 ($3\ s.f$) - similar results but confirming the theory of the curse of dimensionality. This indicates that the third feature R does not provide much additional relevant information that helps the model to make more accurate predictions.

Figure 14: R vs. OBP scatter plot graph

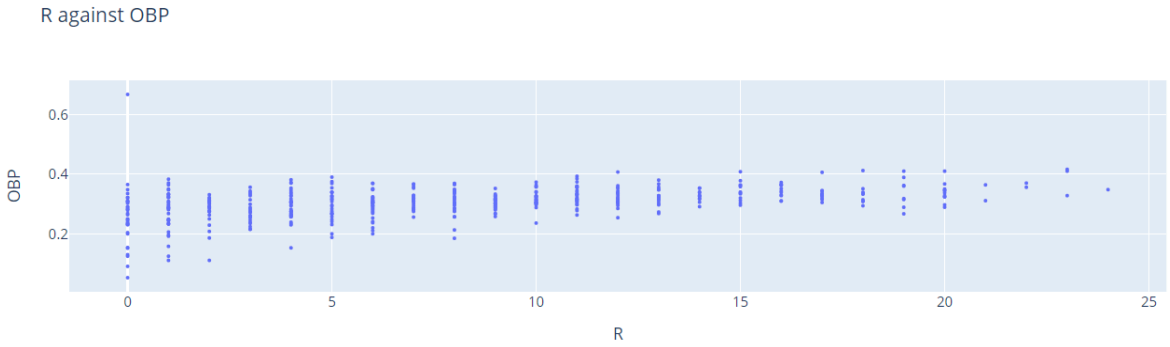

Diagram made by candidate

To verify that, the range of OBP remains relatively constant across different values of R. As the number of runs increases to higher values (> 1), the OBP range

stabilizes and becomes more consistent. This pattern suggests that R does not significantly impact OBP, thus it may be an irrelevant feature. This explains why the MSE values for 2 features and 3 features are close, demonstrating that additional irrelevant features may not considerably affect the model's performance.

Evaluation of the study

Overall, no matter the input features, the MSE value for the most optimal K is always close to 0. Hence, I would say that the model's performance is excellent, with no problem of overfitting or underfitting, therefore leading to a strong conclusion. This high accuracy may result from the direct cause-and-correlation relationship between the selected features and OBP, though, as these features are used in its calculation. Indeed, this outcome does not entirely align with the findings of a similar study that analyzed weighted on-base average (wOBA) given the ball's launch speed and angle, where the model achieved a 76% accuracy (*Nestico, 2024*), confirming the possibility that bias might have been involved in this investigation. Indeed, it is important to note that all algorithms are biased in some form, and the KNNR algorithm is no exception. Despite this inherent bias, the low MSE indicates that for this particular case, the KNNR algorithm has managed to achieve a good trade-off between bias and variance.

Was KNNR an effective choice for this problem?

If my goal is to achieve a high accuracy model, which is often demanded by sports teams, KNNR proves to be an effective choice given its performance. However, if I were to incorporate more offensive features or deal with a very large number of

players, the computational time ($O(n^k)$ where k is the number of features) increases exponentially and becomes a limiting factor, affecting its applications.

There are also alternative algorithms that could have been used. Support Vector Machines (SVM) and neural networks, for instance, might offer different advantages. SVM could provide better performance in high-dimensional spaces, making it ideal if I wanted to add more input features to see how they affect the OBP *(Dhiraj, 2020)*. Neural networks are best for capturing intricate patterns in large datasets *(Simplilearn, 2019)*, which would be useful if I wanted to combine temporary statistics with all available past data, resulting in a huge dataset.

In conclusion, while KNNR has demonstrated excellent performance for this study, it is important to consider the main objective: predicting the OBP of MLB players. Thus, I believe that KNNR was indeed an effective choice for this specific study.

Limitations

One month of data might not be sufficient to determine the overall performance of a player due to various factors such as injuries. Thus, teams might not get the whole picture of a player just from statistics in a month. Additionally, player performance can be highly variable from month to month, with short-term hot streaks or slumps potentially leading to misleading predictions. Hence, even though the model had accurate predictions, teams can not guarantee the consistency of a player throughout the season. To improve the accuracy and reliability of the predictions, future studies

could consider incorporating data from a longer period such as 3 months instead of 1 to ensure performance stability.

Next, the elbow method has its own downfalls because the choice of performance metric (MSE) can influence the elbow point. Different metrics might suggest different optimal K values, leading to potential inconsistency. A better way to evaluate the best K-value I've looked at is cross-validation as it allows for evaluation using multiple performance metrics over multiple folds, reducing bias (Sharma, 2017). However, due to word count limitations, I focused only on the essentials.

Further applications

In team sports, KNNR can be applied to optimize team lineups by analyzing the performance of different player combinations. Furthermore, it can be used in scouting to identify young talents who show similar early-career statistics to successful players. By exploiting various applications, sports teams and analysts can make more supported decisions, thus leading to better team performance.

Ethical and social implications

The unauthorized use of players' data without consent raises significant ethical concerns under the GDPR, which emphasizes data privacy and ownership. Furthermore, the analysis by AI can lead to psychological impacts, such as performance

anxiety, making it crucial to ensure players give informed consent to protect both their privacy and well-being.

Conclusion

In conclusion, this study addressed the research question by utilizing the KNNR model to predict the end-season OBP of MLB players based on their statistics from April. First, through the elbow method, it was determined that the optimal number of neighbors (K) for the KNNR model is 3. This K value provides the best balance between bias and variance, minimizing both overfitting and underfitting. This finding is crucial as it is the performance determinant of the model. Then, due to the Curse of Dimensionality, the analysis identified that AVG and SLG were consistently found to be the most influential in the model's performance. This highlights the importance of selecting the right features, as focusing on the most relevant features enhances the model's predictive accuracy. Lastly, when comparing the MSE of the model using two features (AVG and SLG) against the one using three features (AVG, SLG, and R), it was found that the inclusion of R did not significantly impact the model's accuracy. This indicates that R is not a critical feature for predicting OBP in this context. This finding shows the need for feature relevance and the benefit of simplifying the model by excluding purposeless features.

Based on these findings, the KNNR model can effectively predict an MLB player's end-season OBP by using early-season statistics, helping teams in their decision-making processes. As AI continues to evolve, its applications in sports are

expected to become even more impactful, providing deeper understandings outside of our reach.

Works cited

- Nestico, Thomas. “Modelling XwOBA (with KNN) - Thomas Nestico - Medium.” Medium, Medium, 14 Feb. 2024, medium.com/@thomasjamesnestico/modelling-xwoba-with-knn-9b004e93861a. Accessed 10 Sept. 2024.
- Cclapp. “The Development of Baseball Analytics - Cclapp - Medium.” *Medium*, Medium, 23 Nov. 2023, medium.com/@cclapp_49252/the-development-of-baseball-analytics-86e6bacc8570#:~:text=It%20is%20impossible%20to%20exaggerate. Accessed 14 Aug. 2024.
- Bansal, Shubham. “Supervised and Unsupervised Learning.” *GeeksforGeeks*, 17 Apr. 2019, www.geeksforgeeks.org/supervised-unsupervised-learning/.
- Bechtold, Taylor. “State of Analytics: How the Movement Has Forever Changed Baseball – for Better or Worse.” *Stats Perform*, www.statsperform.com/resource/state-of-analytics-how-the-movement-has-forever-changed-baseball-for-better-or-worse/.
- GeeksforGeeks. “K-Nearest Neighbours - GeeksforGeeks.” *GeeksforGeeks*, 13 Nov. 2018, www.geeksforgeeks.org/k-nearest-neighbours/.
- IBM. “What Is Supervised Learning?” *IBM*, 2023, www.ibm.com/topics/supervised-learning.
- Baseball Reference. “MLB Stats, Scores, History, & Records | Baseball-Reference.com.” *Baseball-Reference.com*, www.baseball-reference.com/.

- Roy, Rahul. “Best Python Libraries for Machine Learning.” *GeeksforGeeks*, 18 Jan. 2019, www.geeksforgeeks.org/best-python-libraries-for-machine-learning/.
- CrashCourse. “Crash Course Artificial Intelligence Preview.” *YouTube*, 2 Aug. 2019, www.youtube.com/watch?v=GvYYFloV0aA&list=PL8dPuuaLjXtO65LeD2p4_Sb5XQ51par_b. Accessed 17 Feb. 2021.
- freeCodeCamp.org. “Machine Learning for Everybody – Full Course.” *YouTube*, 26 Sept. 2022, www.youtube.com/watch?v=i_LwzRVP7bg. Accessed 13 Mar. 2023.
- “KNN Algorithm in Machine Learning | KNN Algorithm Using Python | K Nearest Neighbor | Simplilearn.” *Www.youtube.com*, www.youtube.com/watch?v=4HKqjENq9OU.
- GeeksforGeeks. “K-Nearest Neighbours - GeeksforGeeks.” *GeeksforGeeks*, 13 Nov. 2018, www.geeksforgeeks.org/k-nearest-neighbours/.
- “How to Avoid Overfitting? | R-Bloggers.” *R-Bloggers*, 6 Sept. 2022, www.r-bloggers.com/2022/09/how-to-avoid-overfitting/#google_vignette. Accessed 30 Sept. 2024.
- Soni, Anuuz. “Advantages and Disadvantages of KNN.” *Medium*, 3 July 2020, medium.com/@anuuz.soni/advantages-and-disadvantages-of-knn-ee06599b9336.
- “Standard Stats | Glossary.” *MLB.com*, www.mlb.com/glossary/standard-stats.
- “ML | Bias vs Variance.” *GeeksforGeeks*, 6 Feb. 2020, www.geeksforgeeks.org/bias-vs-variance-in-machine-learning/.

- Stewart, Ken. “Mean Squared Error (MSE) | Definition, Formula, Interpretation, & Facts | Britannica.” *Www.britannica.com*, 22 Mar. 2023, www.britannica.com/science/mean-squared-error.
- Saji, Basil. “K Means Clustering | K Means Clustering Algorithm in Machine Learning.”
- *Analytics Vidhya*, 20 Jan. 2021, www.analyticsvidhya.com/blog/2021/01/in-depth-intuition-of-k-means-clustering-algorithm-in-machine-learning/.
- Super Data Science. “The Elbow Method Explained in Less than 5 Minutes.” *YouTube*, 21 Feb. 2023, www.youtube.com/watch?v=ht7geyMAFfA. Accessed 7 July 2024.
- “Curse of Dimensionality | Curse of Dimensionality in Machine Learning.” *Analytics Vidhya*, 1 Apr. 2021, www.analyticsvidhya.com/blog/2021/04/the-curse-of-dimensionality-in-machine-learning/.
- “K-Nearest Neighbors and Curse of Dimensionality.” *GeeksforGeeks*, 4 Mar. 2024, www.geeksforgeeks.org/k-nearest-neighbors-and-curse-of-dimensionality/.
- Udacity. “Curse of Dimensionality Two - Georgia Tech - Machine Learning.” *YouTube*, 23 Feb. 2015, www.youtube.com/watch?v=OyPcbeiwps8. Accessed 14 Aug. 2024.
- Dhiraj, K. “Top 4 Advantages and Disadvantages of Support Vector Machine or SVM.” *Medium*, 25 Sept. 2020,

dhirajkumarblog.medium.com/top-4-advantages-and-disadvantages-of-support-vector-machine-or-svm-a3c06a2b107.

- Simplilearn. “Neural Network in 5 Minutes | What Is a Neural Network? | How Neural Networks Work | Simplilearn.” *YouTube*, 19 June 2019, www.youtube.com/watch?v=bfmFfD2RIcg.
- Sharma, Abhishek. “Cross Validation in Machine Learning.” *GeeksforGeeks*, 21 Nov. 2017, www.geeksforgeeks.org/cross-validation-machine-learning/.

Appendix

Libraries used (freeCodeCamp.org):

- **Pandas**: Data manipulation and analysis. In this study, it is used to facilitate the data preprocessing process. It offers tools to structure the CSV files into a dataframe for the program to access; to clean, reshape, and merge datasets; and to support operations like filtering datasets.
- **NumPy**: Numerical computing by providing support for arrays, matrices, and a wide range of mathematical functions. It is used in this study to carry out calculations and array manipulation.
- **Matplotlib and Plotly**: Creates static, interactive, and animated visualizations in Python. This consists of various types of customizable graphs. It is used in this investigation to evaluate the KNNR algorithm and to show data distribution.
- **Scikit-learn**: Provides simple and efficient tools for data analysis. This consists of the machine learning algorithms and tools to build the supervised model such

as splitting training and testing datasets. It is used in this study to build the supervised model and to evaluate it.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.graph_objs as go
import plotly.express as px
import itertools

from collections import Counter
from matplotlib.colors import ListedColormap
from sklearn.model_selection import KFold
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import confusion_matrix
from sklearn.metrics import f1_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import mean_squared_error
%matplotlib inline
# Load the april OBP CSV
april_stats = pd.read_csv(r'C:\College\1 IB\EE\april.csv', encoding='cp861', sep=';')
```

```
# Load the end-of-season OBP CSV

end_season_stats = pd.read_csv(r'C:\College\1 IB\EE\Fullseason.csv',
encoding='cp861', sep=';')


#Checking for name duplicates and remove them

april_stats = april_stats.drop_duplicates(subset=['Name'])

end_season_stats = end_season_stats.drop_duplicates(subset=['Name'])


#Finding the names that april_stats and end_season_stats share

common_players = set(april_stats['Name']).intersection(set(end_season_stats['Name']))


# Filter both DataFrames to only include the common players

april_stats_filtered = april_stats[april_stats['Name'].isin(common_players)]

end_season_stats_filtered =
end_season_stats[end_season_stats['Name'].isin(common_players)]


#Select the columns to merge from both datasets to form the final merged dataset

selected_columns_april = ['Name', 'Team', 'G', 'PA', 'HR', 'R', 'RBI', 'SB', 'BB%', 'K%',
'ISO', 'BABIP', 'AVG', 'SLG', 'wOBA', 'wRC+', 'BsR', 'Off', 'Def', 'WAR']

april_stats_selected = april_stats_filtered[selected_columns_april]

end_season_stats_selected = end_season_stats_filtered[['Name', 'OBP']]


#Merging the datasets on players' names

merged_stats = pd.merge(april_stats_selected, end_season_stats_selected,
on='Name')

merged_stats = merged_stats[merged_stats['OBP'] != 0]

merged_stats.reset_index(drop=True)
```

```
#Distribution analysis

#Histogram of OBP

# Creating the histogram

fig = px.histogram(merged_stats, x='OBP', nbins=20, title='OBP Distribution',
labels={'OBP': 'OBP'})


# Displaying the plot

fig.show()


#Remove the 0s from the dataset

merged_stats = merged_stats[merged_stats['OBP'] != 0]

merged_stats = merged_stats.reset_index(drop=True)

Merged_stats


#Histogram of SLG

fig = px.histogram(merged_stats, x='SLG', nbins=20, title='SLG Distribution',
labels={'SLG': 'SLG'})

fig.show()


#Histogram of AVG

fig = px.histogram(merged_stats, x='AVG', nbins=20, title='AVG Distribution',
labels={'AVG': 'AVG'})

fig.show()


#Histogram of R

fig = px.histogram(merged_stats, x='R', nbins=20, title='R Distribution', labels={'R': 'R'})
```

```
fig.show()


#Building the model

#Splitting the dataset into a training set and a testing set

X = merged_stats[['AVG', 'SLG', 'R']]

y = merged_stats['OBP']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)


#Scaling

scaler_X = StandardScaler()

X_train = scaler_X.fit_transform(X_train)

X_test = scaler_X.transform(X_test)


#Define model

regressor = KNeighborsRegressor(n_neighbors=4, p=2, metric ='euclidean') #p=2 is
using euclidean distance and K=4

regressor.fit(X_train, y_train)


#Make predictions on the test set

y_pred = regressor.predict(X_test)


#Evaluating the model

mse = mean_squared_error(y_test, y_pred)

mse


#Creating a function for predicting new values with input k
```

```
def get_error(k):
    #Split dataset
    X = merged_stats[['AVG', 'SLG', 'R']]
    y = merged_stats['OBP']
    X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.2)

    #Scaling
    scaler_X = StandardScaler()
    X_train = scaler_X.fit_transform(X_train)
    X_test = scaler_X.transform(X_test)

    #Defining the model
    regressor = KNeighborsRegressor(n_neighbors=k, p=2, metric ='euclidean') #p=2 is
using euclidean distance
    regressor.fit(X_train, y_train)

    #Making predictions
    y_pred_test = regressor.predict(X_test)
    y_pred_train = regressor.predict(X_train)
    mse_test = mean_squared_error(y_test, y_pred_test)
    mse_train = mean_squared_error(y_train, y_pred_train)
    return (mse_test, mse_train)
#Iterating with different values of k
test_pred = []
train_pred = []
k = []
```

```
for i in range(1, 20):
    errors = get_error(i)
    #Adding the values to the lists
    test_pred.append(errors[0])
    train_pred.append(errors[1])
    k.append(i)


#Plotting a graph of MSE against K
plt.plot(k, test_pred, label='Test MSE')
plt.plot(k, train_pred, label='Train MSE')
plt.xlabel('K value')
plt.ylabel('Mean Squared Error')
plt.title('K-Nearest Neighbors Regressor Performance')
plt.grid(True)
plt.legend()
plt.show()


#After analyzing, the best value of K is 3, and the MSE value is
regressor = KNeighborsRegressor(n_neighbors=3, p=2, metric ='euclidean') #p=2 is
using euclidean distance
regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)
mse_3 = mean_squared_error(y_test, y_pred)
mse_3
```

```
#Therefore, lets try to test the MSE on other combinations of features of 3 using the best K value

#Getting all combinations of 3 that is relevant to the target variable OBP
combinations = list(itertools.combinations(['HR', 'R', 'RBI', 'SB', 'AVG', 'SLG'], 3))
print(combinations)


#Finding the MSE of each combination
#Defining a function with the best k value gotten from before
def get_mse(combination):
    #Split dataset
    X = merged_stats[combination]
    y = merged_stats['OBP']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    #Scaling
    scaler_X = StandardScaler()
    X_train = scaler_X.fit_transform(X_train)
    X_test = scaler_X.transform(X_test)

    #Defining the model
    regressor = KNeighborsRegressor(n_neighbors=3, p=2, metric ='euclidean') #p=2 is using euclidean distance
    regressor.fit(X_train, y_train)

    #Making predictions
    y_pred_test = regressor.predict(X_test)
    mse_test = mean_squared_error(y_test, y_pred_test)
    return mse_test

#Find the MSE
combinations_mse = []
lowest_mse_combinations = []
for combo in combinations:
    temp_mse = get_mse(list(combo))
```

```
    combinations_mse.append(temp_mse)
print(combinations_mse)

#Where is it the minimum?
print(min(combinations_mse))
combination_index = combinations_mse.index(min(combinations_mse))
lowest_mse_combinations.append(combinations[combination_index])
print(lowest_mse_combinations)

#Now, I want to find out which features appears the most in the combinations with the lowest MSE
#Creating the function
def get_frequent_feature():
    combinations_mse = []
    for combo in combinations:
        temp_mse = get_mse(list(combo))
        combinations_mse.append(temp_mse)
    combination_index = combinations_mse.index(min(combinations_mse))
    lowest_mse_combinations.append(combinations[combination_index])

#Iterating 60 times to see which 2 features pops up the most

for i in range(0, 60):

    get_frequent_feature()


#Counting the frequency of each feature in the triplets and getting the two most frequent features

# Count the frequency of each feature in the triplets

feature_counter = Counter()

for triplet in lowest_mse_combinations:

    feature_counter.update(triplet)
```

```
# Get the two most frequent features
most_common_features = feature_counter.most_common(2)
print(most_common_features)

#Create a pie chart to show the trend of each feature
for triplet in lowest_mse_combinations:
    feature_counter.update(triplet)

# Extract the features and their counts
features = list(feature_counter.keys())
counts = list(feature_counter.values())

# Create a bar chart for the frequencies of all features
fig, ax = plt.subplots()
ax.bar(features, counts, color='skyblue')
ax.set_xlabel('Features')
ax.set_ylabel('Frequency')
ax.set_title('Frequency of Features in Lowest MSE Combinations')
ax.set_xticks(features)
ax.set_xticklabels(features)

plt.show()
#Therefore these are the features with the most significant contributions
#Graphing the data distribution
x_data = merged_stats['AVG']
```

```
y_data = merged_stats['SLG']
color = merged_stats['OBP']

slope, intercept = np.polyfit(x_data, y_data, 1)
line_of_best_fit = slope * x_data + intercept

# Create the 2D scatter plot
fig = px.scatter(
    merged_stats,
    x='AVG',
    y='SLG',
    color='OBP',
    color_continuous_scale='Plotly3',
    title='SLG against AVG',
    labels={'AVG': 'AVG', 'SLG': 'SLG'}
)

fig.show()

#Now, what happens to the model's MSE when we only use these 2 features?
#Split dataset
X = merged_stats[['AVG', 'SLG']]
y = merged_stats['OBP']
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, test_size=0.2)
```

```
#Scaling

scaler_X = StandardScaler()

X_train = scaler_X.fit_transform(X_train)

X_test = scaler_X.transform(X_test)


#Defining the model

regressor = KNeighborsRegressor(n_neighbors=6, p=2, metric ='euclidean') #p=2 is
using euclidean distance

regressor.fit(X_train, y_train)


#Making predictions

y_pred_test = regressor.predict(X_test)

mse_2 = mean_squared_error(y_test, y_pred_test)


print("2 features:", mse_2)

print("3 features:", mse_3)


#MSE for 3 features is lower than the MSE for 2 features
#R vs OBP scatter plot

x_data = merged_stats['R']

y_data = merged_stats['OBP']


slope, intercept = np.polyfit(x_data, y_data, 1)

line_of_best_fit = slope * x_data + intercept


# Create the 2D scatter plot
```

```
fig = px.scatter(
    merged_stats,
    x='R',
    y='OBP',
    title='R against OBP',
    labels={'R': 'R', 'OBP': 'OBP'}
)

fig.update_traces(marker=dict(size=3),
                  selector=dict(mode='markers'))

fig.show()
```